SIMATS ENGINEERING

PYTHON PROGRAMMING ASSIGNMENT 2

Control Flow, Strings and Functions

Student	Varshan J S
Register Number	1924260130A
Institution	SIMATS Engineering
Python Version	Python 3.14.2
GitHub Repository	CSA08-1924260130A-Assignment2

Assignment Coverage

Control flow constructs, loops, string operations, functions, parameters, scope, recursion, memoisation, algorithm analysis and redesign validation.

This individual assignment is supported by Python source files and task execution evidence maintained in the linked GitHub repository.

Q1 – Decisions with Many Boundaries (6 Marks)

Problem. Determine electricity subsidy eligibility using consumption units, ration-card status and the presence of a household member above 80 years of age.

Test case	Expected decision
80 units	Full subsidy
100 units	Full subsidy
101 units + ration card	Partial subsidy
150 units, no ration card	No subsidy
200 units + ration card	Partial subsidy
201 units, no senior member	No subsidy
250 units + member above 80	Special subsidy
250 units, no member above 80	No subsidy

Analysis. Independent if statements are evaluated separately, so overlapping conditions can produce more than one action. An if–elif–else chain selects only the first matching branch and is clearer when exactly one decision is required. For fixed rules, time and auxiliary space are O(1). A lookup list searched for k slabs takes O(k) time and O(k) storage.

Design decision. Use an if–elif–else chain with ordered, mutually exclusive ranges to avoid duplicate subsidy decisions.

Q2 – Loops that Count Attempts (4 Marks)

Problem. Verify an OTP with three attempts. Blank entries do not count; a correct OTP ends verification; three wrong attempts lock the account for 30 minutes. A correct security-question answer adds two attempts.

Loop design. A while loop suits an interactive process whose limit may change. A for loop with range(3) suits a fixed three-attempt policy. After successful security verification, increase the limit to five.

Control statements. continue skips blank entries without incrementing the failed-attempt count; break exits after a correct OTP; pass is a no-operation placeholder; loop else runs when the loop finishes without break and can trigger lockout.

Common bug. Incrementing the counter before checking for a blank entry incorrectly counts blank submissions. Increase the counter only after a non-blank incorrect OTP.

Complexity. For a attempts, time is O(a) and auxiliary space is O(1). A recursive handler also takes O(a) time but uses O(a) call-stack space, so an iterative loop is preferable.

Q3 – Vehicle Registration Number Validator (6 Marks)

Problem. Normalise and validate vehicle registration strings using string operations and slicing, without regular expressions.

Normalisation. Remove spaces and hyphens, then convert to uppercase. For example, tn 09 ab 1234 becomes TN09AB1234. A valid registration contains two state letters, two district digits, one or two series letters and four final digits. Receipt format: TN 09 AB 1234.

Input	Normalised	Result
tn 09 ab 1234	TN09AB1234	Valid
KA-01-M-0007	KA01M0007	Valid
TN9AB1234	TN9AB1234	Invalid
TN 22 1234 AB	TN221234AB	Invalid

String concepts. Python strings are immutable. upper() and replace() return new strings; they do not modify the original. For a 9-character value, s[-4:] selects the final four digits and s[4:-4] selects one series letter. For a 10-character value, these slices select the final four digits and two series letters.

Complexity. Normalisation and validation take O(L) time for input length L. A regular-expression approach is also O(L), but the string-method implementation is explicit and easy to trace.

Q4 – Building Long Text: + or join? (6 Marks)

Method. The same newline-separated statement was constructed using repeated +=, a list followed by join(), and generated f-string lines. The benchmark was run locally on Python 3.14.2; each time is the average of three runs.

Lines	Repeated +=	join()	f-string	Output chars
1,000	0.000343 s	0.000016 s	0.000243 s	13,690
10,000	0.002132 s	0.000327 s	0.001608 s	146,890
200,000	0.412845 s	0.006131 s	0.039803 s	3,248,890

Observed result. At 200,000 lines, join() took 0.006131 seconds, versus 0.412845 seconds for repeated += and 0.039803 seconds for f-string lines. In this run, join() was approximately 67.3 times faster than += and 6.5 times faster than f-string line generation.

Analysis. Strings are immutable, so repeated concatenation can cause repeated allocation and copying and may approach $O(N^2)$ total work. join() combines collected parts in O(N) time relative to total output length. CPython may optimise some += patterns, so timings depend on machine and workload. For very large statements, file streaming or StringIO can avoid retaining all parts and the final string simultaneously.

Q5 – Functions, Parameters and Scope (7 Marks)

Mutable default bug. In add_item_buggy(item, order=[]), the default list is created once when the function is defined. Calls omitting order reuse that list, so one customer's items can appear in the next customer's order. The corrected function uses order=None and creates a fresh list inside each call.

Parameters and return values. Positional arguments are matched by order; keyword arguments identify parameters by name; defaults are used when values are omitted. A keyword-only GST rate makes billing calls explicit. A function can return subtotal, GST and total together as a tuple.

Scope. Local variables belong to a function. A global variable belongs to module scope and requires global when reassigned inside a function. nonlocal reassigns a variable in the nearest enclosing function scope. Global state is generally avoided because it creates hidden dependencies; parameters and return values are easier to test.

Testing and complexity. Three customers called in sequence should receive independent order lists. Appending is O(1) amortised; an order with m items uses O(m) storage.

Q6 – Recursion and Its Limits (8 Marks)

Problem. Count the ways to climb n temple steps when each move covers one or two steps. Let W(n) be the number of ways to reach step n. Base cases: $W(0)=1$ and $W(1)=1$. For $n \geq 2$, $W(n)=W(n-1)+W(n-2)$.

n	1	2	3	4	5	6
W(n)	1	2	3	5	8	13

Analysis. Plain recursion repeats overlapping calculations and has exponential time complexity. Memoisation stores computed states, reducing time to O(n), with O(n) cache space and O(n) recursive stack depth. The memoised version can still raise RecursionError for large n because Python limits recursive call depth.

Deployment decision. Raising the recursion limit is not a robust fix. An iterative method computes the same recurrence in O(n) time while retaining only the previous two values, using O(1) auxiliary space.

Q7 – Diagnose, Redesign and Validate (13 Marks)

Selected limitation. Q6 memoisation avoids repeated calculations but still uses O(n) call-stack depth. For sufficiently large n, it may fail with RecursionError even though the number of distinct states is linear.

Objective. Compute the same number of climbing paths for n=1,000 and larger without recursion, using constant auxiliary memory.

Redesign. Use bottom-up iteration and retain only the previous two results. Initialise a=1 and b=1. For each step from 2 through n, calculate current=a+b, then shift b=a and a=current. Return a. If n≤1, return 1.

Pseudocode

```
if n <= 1: return 1
a = 1; b = 1
for step from 2 to n:
    current = a + b
    b = a; a = current
return a
```

Prediction and validation. The iterative result should match memoised recursion for inputs where recursion completes. For n=1,000, iteration should complete without recursion-depth failure. A suitable validation set is n=1, 2, 5, 10, 25, 100 and 1,000.

Trade-off. Both memoisation and iteration take O(n) time. Memoisation uses O(n) cache and O(n) stack space; the iterative redesign uses O(1) auxiliary space. The iterative method is the more suitable deployment choice for large inputs.

Overall Complexity Summary

Question	Time	Auxiliary space
Q1 – Decision branches	O(1) for fixed rules	O(1)
Q2 – OTP attempts	O(a)	O(1)
Q3 – Registration validation	O(L)	O(L) for normalised string
Q4 – join() construction	O(N)	O(N) output/parts
Q5 – Add item	O(1) amortised per append	O(m) per order
Q6 – Memoised recursion	O(n)	O(n) cache and stack
Q7 – Iterative redesign	O(n)	O(1)

Conclusion

This assignment applies Python control flow, loops, string processing, functions, scope and recursion to practical problems. The analysis demonstrates the importance of mutually exclusive decision branches, correct loop-counter placement, immutable string behaviour, safe handling of mutable defaults and awareness of recursion limits. The recorded benchmark shows join() as the fastest of the three tested text-building methods for the largest input on the measured system. The Q7 redesign preserves the recurrence while avoiding recursion-depth limits and reducing auxiliary space to O(1).

Source code and task execution evidence are maintained in the GitHub repository listed on the cover page.